

RESIBO, PLEASE

Midterm Project Documentation

Resibo, Please: An AI-Assisted Mobile Civic Decision Simulation Game for Voter Literacy and SDG-Based Decision-Making

Student / Group	[To be supplied before submission]
Course / Section	[To be supplied before submission]
Application Version	1.0.0+1
Repository State	Branch v1.0 • commit 561e7d6
Document Date	September 2, 2026

All politicians, cities, parties, media outlets, and events in this project are fictional.

The application does not provide real election guidance or candidate recommendations.

Table of Contents

Table of Contents 2
1. Project Title and Description 3
1.1 Project identity 3
1.2 Project description 3
1.3 Educational purpose and SDG alignment 3
2. Project Objectives 4
2.1 General objective 4
2.2 Specific objectives 4
3. Scope and Limitations 4
3.1 Scope 4
3.2 Limitations 5
4. System Architecture 6
5. System Flowchart 8
6. Database Design / ERD 9
6.1 Online database 9
6.2 Current table schema 9
6.3 API operations 10
6.4 Data minimization and access control 10
7. Data and Component Architecture 11
8. Project Progress Report 13
8.1 Current overall status 13
8.2 Progress by area 13
8.3 Major completed features 14
8.4 Features currently being refined 14
8.5 Remaining work after the midterm 14
8.6 Testing and stability status 15
8.7 Repository documentation inconsistencies found 15
9. Summary 16
Appendix A — Selected System Screenshots 17

DOCUMENT COVERAGE

This single file covers all six items under A. Project Documentation: title and description, objectives, scope and limitations, updated architecture/flowchart, database design, and the current progress report.

1. Project Title and Description

1.1 Project identity

Item	Verified project information
Project title	Resibo, Please
Formal title	Resibo, Please: An AI-Assisted Mobile Civic Decision Simulation Game for Voter Literacy and SDG-Based Decision-Making
Primary platform	Android mobile; Flutter Web is also used for development and testing
Technology	Flutter/Dart with GoRouter, Sembast, SharedPreferences, HTTP, Supabase Anonymous Auth/PostgREST, Audioplayers, and a lightweight Flame menu layer
Current delivery	One complete fictional mayoral election followed by one deterministic 365-day term

1.2 Project description

Resibo, Please is a fictional civic and political decision simulation designed to let players practice evidence-based voting in a safe environment. A new local city is created from a seed and configuration, producing four to six active problems and five fictional mayoral candidates. The player studies the city, investigates the candidates, casts a ballot without being shown a recommended answer, and then observes how the elected mayor governs during a simulated 365-day year.

Candidate checking is the central learning experience. Each generated candidate is represented through a Civic Archive biography, campaign promises, structured history, Campaign Trail material, CityBook posts, debate responses, comparative news coverage, public and campaign records, evidence relationships, unresolved questions, and a personal Receipts Board. These views are generated from one saved candidate-truth structure so that claims, records, and consequences remain connected.

After voting, the Living City engine evaluates a complete deterministic year. City indicators, issue severity, promise-based projects, public trust, budget health, corruption pressure, world conditions, and causal event chains evolve from the inherited city and the elected candidate's hidden abilities. The player can observe the city map, event history, Chronicle, News, R-8 television reports, CityBook responses, and the final Term Report.

Local play remains the canonical experience. After a completed term, the player may optionally publish a deliberately sanitized Public or Private city snapshot to a Supabase community gallery. Other players can explore public snapshots, search display names, and manage their own uploaded snapshots; full saves and hidden simulation data are never part of the gallery payload.

1.3 Educational purpose and SDG alignment

Alignment	Project contribution
Primary: SDG 4 — Quality Education	Supports civic education, media and information literacy, critical thinking, evidence evaluation, and reflective decision-making.
Secondary: SDG 11 — Sustainable Cities and Communities	Uses local governance scenarios involving transport, services, resilience, housing-related infrastructure, health, education, water, employment, and public trust.
Related issue domains	City issue records may also reference SDGs 1, 2, 3, 6, 8, and 13 when the generated problem is relevant.

The intended primary users are senior high school students, college students, first-time or young voters, and civic education learners. Teachers and general mobile-game players are secondary audiences. The current game measures decisions inside a fictional simulation; it does not test or predict a real election.

POLITICAL NEUTRALITY

All candidates, parties, cities, records, media organizations, and events are fictional. The game never displays a best-candidate label, hidden quality score, match percentage, or voting recommendation and does not provide official election guidance.

2. Project Objectives

2.1 General objective

To develop a mobile civic decision simulation that strengthens voter, media, and evidence literacy by allowing players to investigate fictional candidates, make an independent electoral decision, and observe the systemic consequences of that decision within a deterministic city simulation.

2.2 Specific objectives

1. Generate varied but reproducible fictional cities containing four to six active problems selected from twelve civic issue families.
2. Generate exactly five distinct fictional mayoral candidates per normal run and maintain consistent identities, histories, promises, evidence, media, and hidden tradeoffs through versioned seeded generation.
3. Provide multiple evidence-facing investigation formats—including biographies, promises, debates, records, CityBook, Chronicle, News, television reports, and a Receipts Board—without exposing a pre-election recommendation.
4. Allow the player to cast and confirm a ballot based on a selected candidate, priority issue, and self-reported confidence.
5. Simulate one complete 365-day administration using bounded indicators, issue state, candidate abilities, projects, world conditions, and procedural events that remain stable after save restoration.
6. Persist up to five local city runs, mutable investigation state, settings, election results, and Living City history using versioned storage and migrations.
7. Provide English and Filipino interface support together with reduced-motion, high-contrast, audio, volume, and critical-event interruption settings.
8. Implement an optional privacy-limited Supabase community feature with explicit REST GET, POST, PATCH, and DELETE operations for player profiles and sanitized city snapshots.
9. Maintain responsive, game-styled interfaces that work on narrow Android screens and provide controlled scrolling instead of clipping on short displays.

3. Scope and Limitations

3.1 Scope

Area	Implemented scope
Application	Flutter application for Android with a Flutter Web development/testing target, responsive custom UI, GoRouter navigation, immersive Android presentation, and branded launcher assets.
City generation	Seeded city setup; 4–6 problems drawn from 12 issue families; 11 tracked city indicators; starting pressure, concern priorities, and trend/severity variation.
Candidate generation	Five unique candidates selected from a seven-portrait pool; compatible fictional names; careers, parties, slogans, hidden tradeoffs, histories, relationships, controversies, promises, and information footprints.
Candidate investigation	Candidate Roster; Bio/Civic Archive; four promises; Then-vs-Now context; Campaign Trail; three-topic Debate; nine evidence records per candidate; dossiers; source readers; bookmarks, pins, notes, costs/unlocks, Receipts Board, and comparison.

Area	Implemented scope
Information ecosystem	Issue-rooted Chronicle and resident voices; candidate and term CityBook posts; three editorial framings of shared events; full News articles; deterministic R-8 TV reports.
Election	Game-styled ballot, candidate/issue/confidence reflection, confirmation, and animated winner reveal.
Living City	One 365-day term with Day/Week/Month/Year controls, 11 indicators, issue state, promise-derived projects, world conditions, events, causal chains, map markers, 27 semantic anchors, 19 persistent landmarks, history, and final report.
Local persistence	Five Sembast save slots with schema-8 migration; generated election snapshot, investigation state, vote, full year history, current day/speed/pause state, inspected events, and settings persistence.
Community API	Optional Anonymous Auth identity, 3–24 character display name, Public/Private city snapshot publishing, Explore, My Cities, player search, privacy changes, deletion, separate caches, and explicit REST CRUD.
Language and audio	English/Filipino localization; reduced motion; high contrast; SFX/dialogue/music toggles and volumes; route-aware music contexts; critical-event interruption settings.

3.2 Limitations

- The content is entirely fictional and cannot predict real elections, score real politicians, or recommend how a person should vote.
- The current loop covers one mayoral election and one 365-day term. Vice Mayor, multiple offices, connected future elections, and long-term administration history are not yet implemented.
- The simulation is aggregate and systemic. It does not run individual citizen, vehicle, building, household, or road-agent artificial intelligence.
- The player observes the elected administration; the current term does not let the player act as mayor or directly allocate the city budget.
- Runtime AI does not determine simulation truth, candidate integrity, evidence, or outcomes. Current variation is produced by deterministic rules and authored content libraries.
- There is no real-time multiplayer, friends/follow system, messaging, social comments between real users, rankings, or shared cloud save.
- Supabase sharing is asynchronous and optional. Clearing app data or changing devices can lose access to an anonymous identity because no email/password recovery system is implemented.
- The deployed Supabase project must still be checked manually for the V2 SQL, Anonymous Sign-Ins, RLS state, and live request behavior; repository tests verify code and intended policy definitions, not dashboard state.
- Music tracks and the reporter dialogue sound require verified redistribution provenance or replacement before a public/commercial release. The application remains usable without optional music.
- The Android classroom APK currently uses debug signing. A private upload/release keystore is required before Play Store or public distribution.
- Some media slots reuse existing art or generic safe fallbacks; the procedural content and visual libraries can be expanded without changing current saved worlds.

4. System Architecture

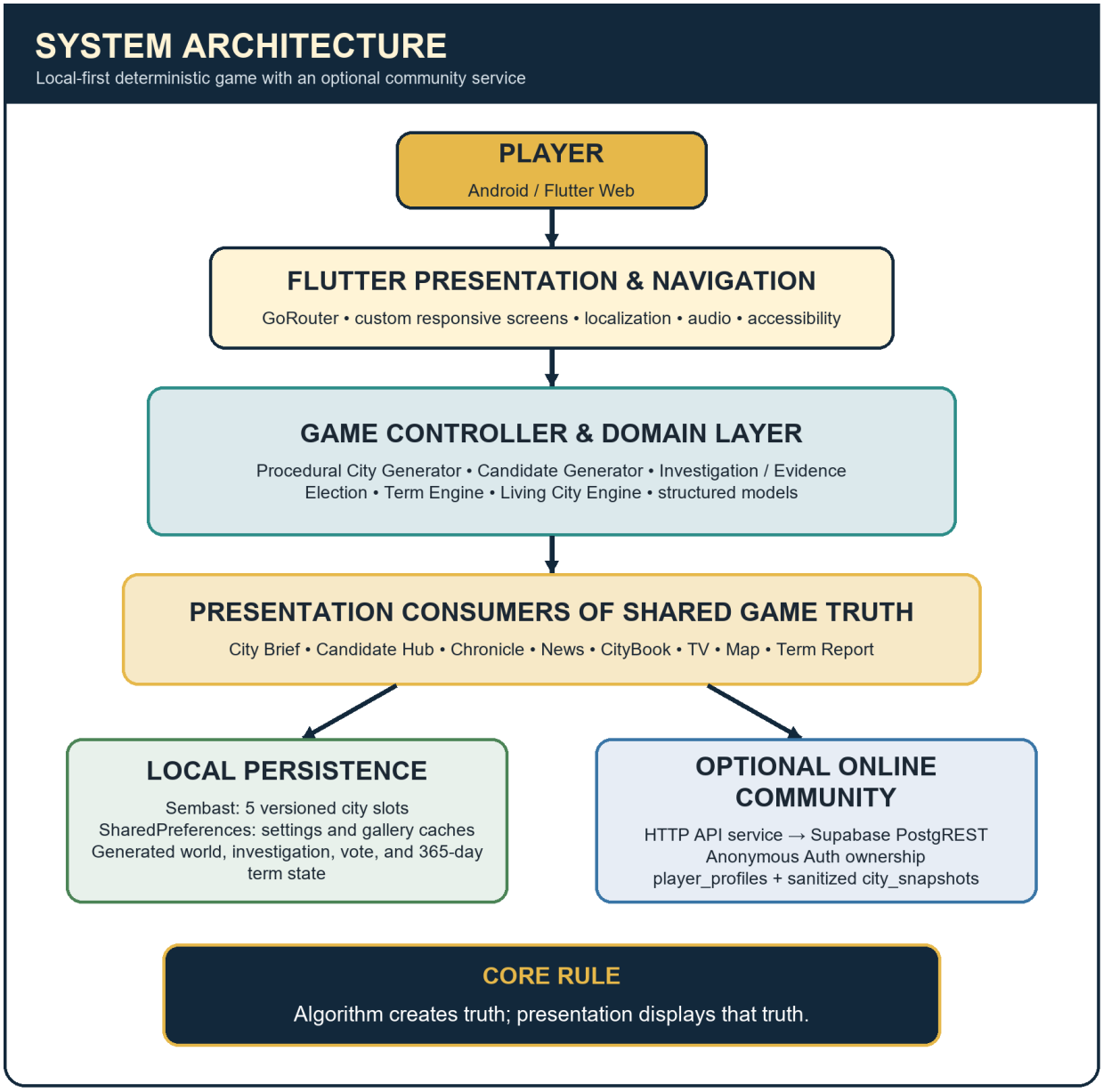


Figure 1. Current Resibo, Please system architecture.

The application follows a local-first layered design. Flutter screens and GoRouter present the game and collect user actions. GameController coordinates the active save and delegates truth generation to deterministic domain engines. Typed models represent the city, candidate, evidence, election, events, and term state. Repositories and codecs persist the resulting state instead of asking widgets to regenerate it.

Layer	Current responsibility	Representative implementation
Presentation	Responsive game screens, navigation, localization, visual states, accessibility, and input	lib/features, lib/app/router.dart, lib/app/theme
Application state	Active save selection, investigation mutations, election handoff, simulation visibility, and save scheduling	lib/core/state/game_controller.dart
Domain	Seeded city/candidate generation, evidence relationships, information ecosystem, election outcome, and Living City rules	lib/domain/models, lib/domain/simulation

Layer	Current responsibility	Representative implementation
Data	Authored libraries, scenario configuration, persistence codecs, gallery service/repository/cache	lib/data
Local storage	Canonical city saves, settings, and read-only gallery cache	Sembast + SharedPreferences
Optional online	Anonymous identity and sanitized community records	Supabase Auth + PostgREST + PostgreSQL/RLS

ARCHITECTURE PRINCIPLE

Algorithm creates truth; presentation displays that truth. Chronicle, News, TV, CityBook, map markers, and the Term Report consume shared saved event or candidate records rather than operating as independent simulation engines.

5. System Flowchart

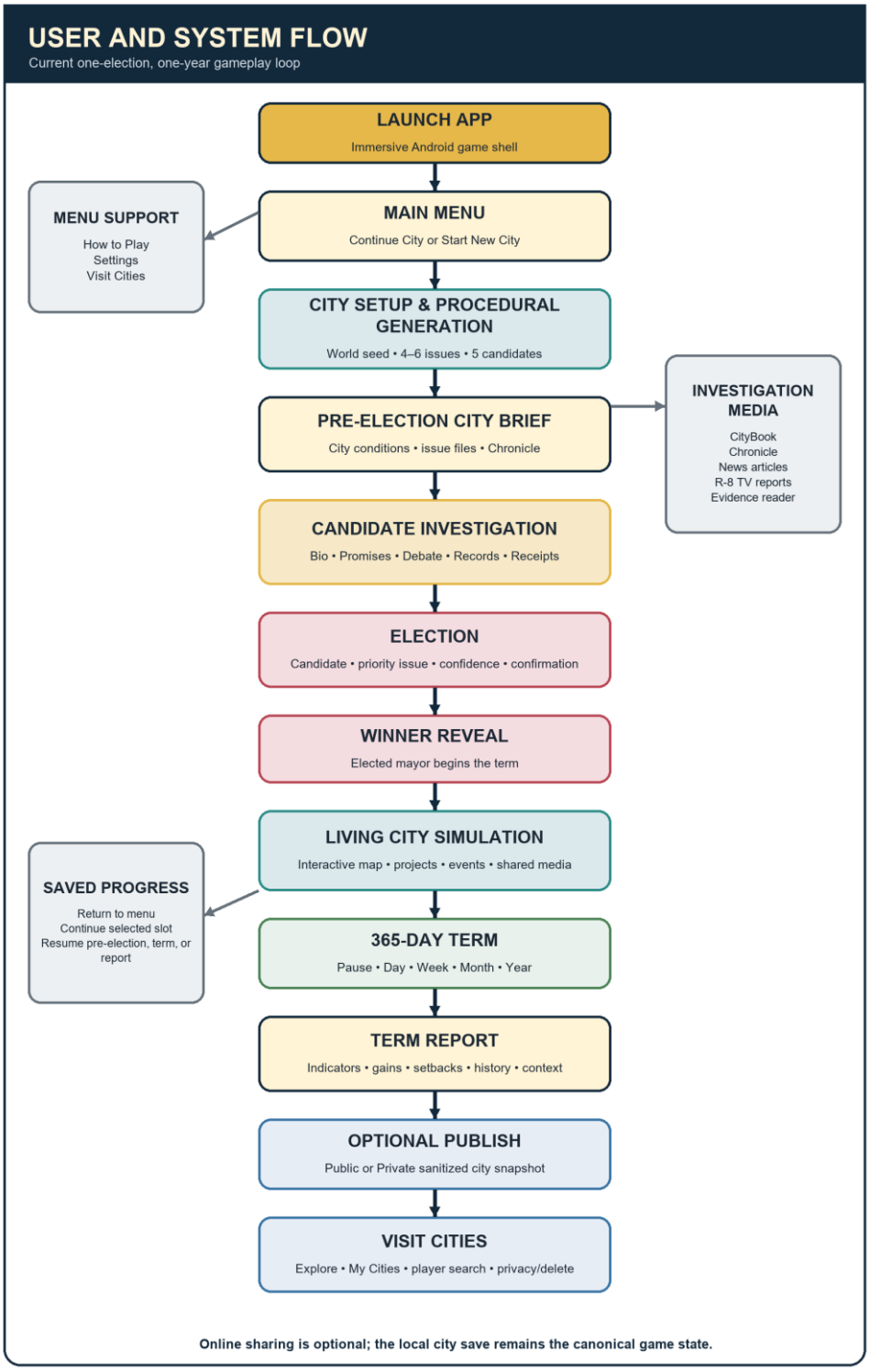


Figure 2. Current player and system flow from launch to optional city sharing.

A player may leave and continue from the current save stage. Before voting, the City, Dossiers, and Chronicle navigation keeps city conditions separate from candidate claims while still allowing cross-links into evidence and media. After voting, the current simulation day determines which events and related Chronicle, News, TV, and CityBook records are visible.

6. Database Design / ERD

6.1 Online database

The optional community feature uses the Supabase REST API (PostgREST) and Supabase Anonymous Auth. The current V2 SQL defines two public tables linked to Supabase-managed auth.users. The anonymous session receives a stable auth.uid() on the current installation, while the player supplies only a display name.

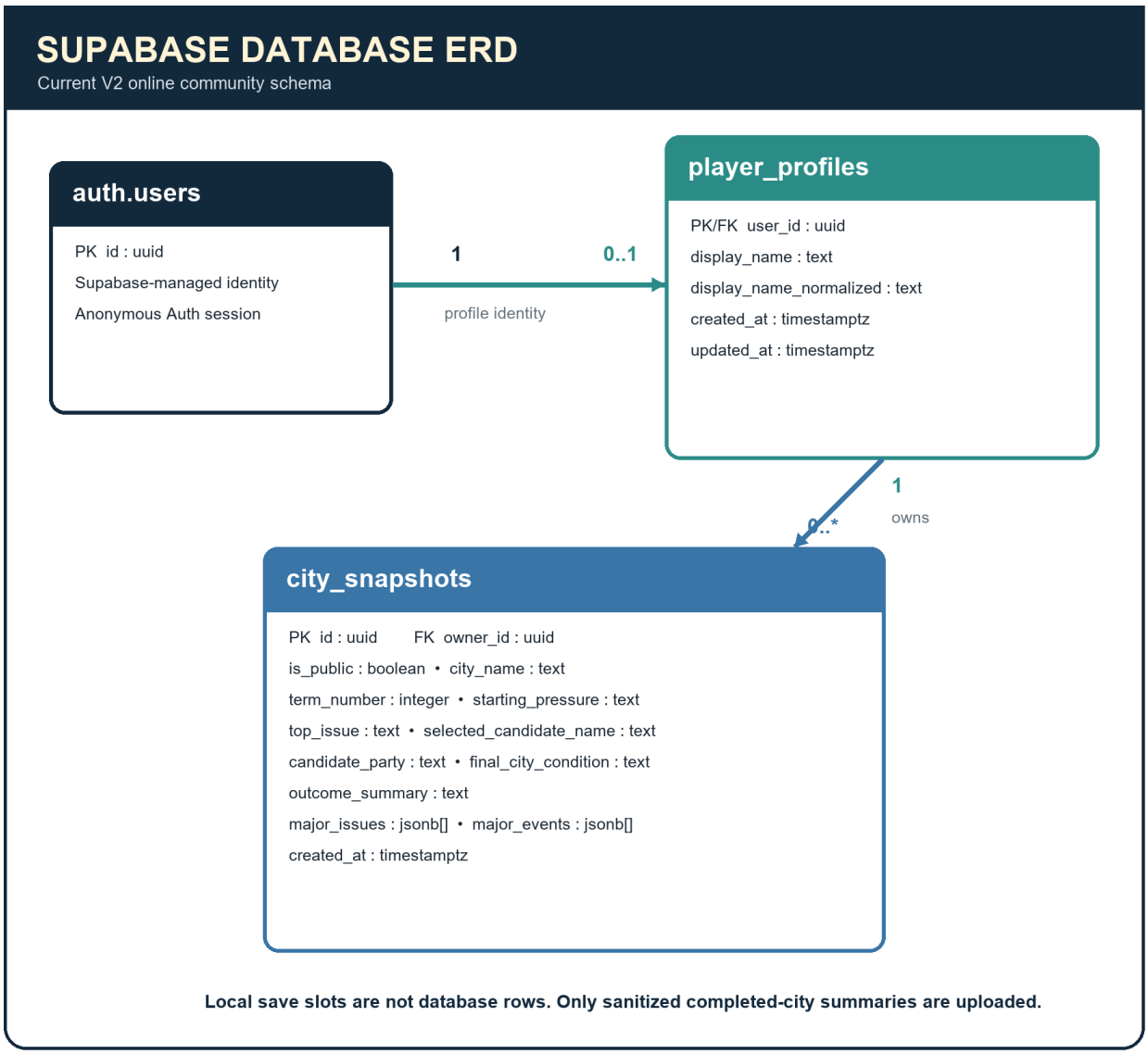


Figure 3. Supabase V2 entity-relationship diagram.

6.2 Current table schema

Table	Field	Type / rule
player_profiles	user_id	uuid; primary key; defaults to auth.uid(); foreign key to auth.users(id); cascade delete
player_profiles	display_name	text; required; trimmed; 3–24 characters; supported letters/numbers/underscore/spaces
player_profiles	display_name_normalized	text; required; unique index; must equal lower(display_name)
player_profiles	created_at / updated_at	timestamptz; server defaults; update trigger maintains

Table	Field	Type / rule
		updated_at
city_snapshots	id	uuid; primary key; generated by gen_random_uuid()
city_snapshots	owner_id	uuid; required; defaults to auth.uid(); foreign key to player_profiles(user_id); cascade delete
city_snapshots	is_public	boolean; required; default true
city_snapshots	city_name / term_number / starting_pressure	bounded text/integer values describing the completed fictional city
city_snapshots	top_issue / selected_candidate_name / candidate_party	bounded text presentation fields
city_snapshots	final_city_condition / outcome_summary	enumerated condition and summary limited to 600 characters
city_snapshots	major_issues / major_events	jsonb arrays; strings only; maximum 8 entries and 2048 bytes each
city_snapshots	created_at	timestamptz; required; default now()

6.3 API operations

Method	Resource	Implemented purpose
GET	/rest/v1/player_profiles	Restore a profile or perform a server-side case-insensitive partial display-name search.
POST	/rest/v1/player_profiles	Create the current anonymous user's validated display-name profile.
GET	/rest/v1/city_snapshots	Load public Explore rows, owned rows, or one searched player's public rows using query filters.
POST	/rest/v1/city_snapshots	Publish one completed sanitized city as Public or Private; id and owner_id are not supplied by the client.
PATCH	/rest/v1/city_snapshots?id=eq.<id>	Change only is_public for a snapshot owned by the current authenticated session.
DELETE	/rest/v1/city_snapshots?id=eq.<id>	Delete an owned uploaded snapshot without deleting the local city save.

6.4 Data minimization and access control

- Local gameplay saves are not stored in Supabase. Sembast remains the canonical storage for world seed, candidates, evidence, investigation state, vote, events, and term history.
- PublicCitySnapshot cannot serialize the world seed, hidden candidate capabilities, complete save document, bookmarks, private notes, or investigation history.
- V2 SQL enables and forces row-level security. Authenticated sessions may read public rows or their own rows; insert/update/delete checks compare owner_id with auth.uid().
- Column grants restrict city-snapshot updates to is_public, while the unauthenticated database role receives no table privileges.
- The SQL file describes intended policy. The actual deployed dashboard must be manually verified before the demonstration.

7. Data and Component Architecture

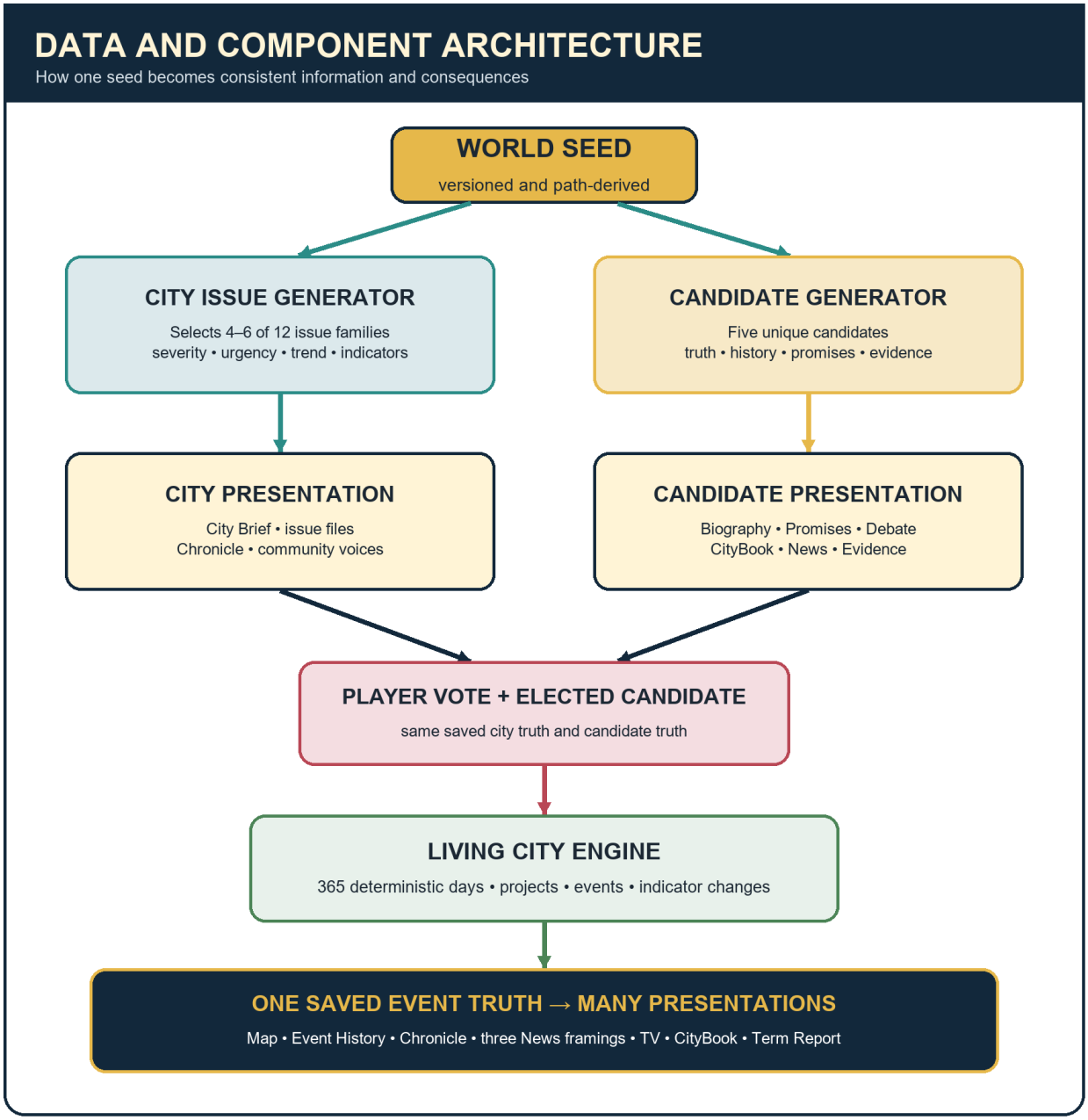


Figure 4. Seeded data flow from city creation to shared information and term outcomes.

SeedDerivation creates isolated subsystem paths, so a random draw added to CityBook does not consume the path used for candidate identity, city issues, another candidate, or the Living City. A normal generated election is serialized after creation and restored from that snapshot instead of being regenerated during navigation. Candidate generation is version 1, the current Living City engine is generation version 4, and the current local save schema is version 8.

Data source	Generated or stored output	Primary consumers
World seed + city configuration	4–6 CityProblem records and 11 starting indicators	City Brief, Chronicle, candidate issue fit, term engine
Candidate truth profile	Identity, capabilities, history, promises, relationships, controversies, evidence graph	Biography, Promises, Debate, CityBook, News, Records, election and term engine
Player investigation	Viewed/unlocked evidence, costs, bookmarks, pins,	Dossier, evidence reader, Receipts

Data source	Generated or stored output	Primary consumers
state	suspicious flags, private notes	Board, save restore
Confirmed election	Selected candidate, top issue, confidence, deterministic term seed	Winner reveal, TermEngine, LivingCityEngine
Saved CityEvent	Date, importance, effects, location, related records, parent chain	Map, Event History, Chronicle, three News framings, TV, CityBook, Term Report
Completed TermResult	Indicator movement, projects, event history, explanation, final condition	Term Report and sanitized gallery snapshot builder

8. Project Progress Report

8.1 Current overall status

Resibo, Please is a functional midterm MVP on branch v1.0. The full local loop—from city creation and candidate investigation through election, a deterministic 365-day administration, and the Term Report—is implemented. The optional Public City Gallery is implemented in code with profile setup, Public/Private publishing, Explore, My Cities, search, privacy changes, deletion, and cached read states. Current work is primarily refinement, deployment verification, content expansion, and public-release preparation rather than construction of a missing core loop.

FRESH VERIFICATION — SEPTEMBER 2, 2026

Static analysis completed with no issues. The complete Flutter test suite passed all 155 tests. Debug and release APK artifacts are present in build/app/outputs/flutter-apk; the classroom release still uses debug signing.

8.2 Progress by area

Area	Status	Current accomplishment
UI/UX	Implemented; refinement	Custom political-cartoon/mobile-game visual system across major screens; responsive Main Menu, safe scrolling, immersive Android layout, and non-default game controls.
Navigation	Complete for MVP	GoRouter connects Main Menu, gallery, City Brief, Chronicle, candidate systems, News, TV, election, simulation, and report routes.
City setup/generation	Complete for MVP	Seven setup choices, deterministic world seed, 4–6 active issues, 12 issue-family library, issue metrics and indicators.
Candidate generation	Complete for MVP	Five-of-seven portraits, unique stable identities, hidden tradeoffs, structured history, promises, CityBook, Debate, News, and 45 evidence records per election.
Candidate investigation	Complete for MVP	Roster, Candidate Hub, Civic Archive, Promises, Campaign Trail, Debate, Dossier, source readers, evidence costs/unlocks, Receipts Board, notes, and comparison.
Information ecosystem	Complete for MVP	Chronicle, resident voices, CityBook, one-event/three-outlet News, full articles, and R-8 reports consume shared records.
Election	Complete	Ballot reflection, confirmation, animated winner reveal, and transfer of selected candidate truth into the term simulation.
Living City simulation	Complete for one term; refinement	365 days, Day/Week/Month/Year controls, event templates, projects, world conditions, causal chains, interactive map, landmarks, markers, history, and visibility gating.
Local persistence	Complete for MVP	Five Sembast slots, schema-8 saves/migrations, generated candidate and term snapshots, settings, and recovery-safe repository behavior.
Settings/localization	Complete for current UI	English/Filipino; reduced motion; high contrast; dialogue/SFX/music toggles and volumes; four critical-event interruption modes.
Audio	Functional; asset review	Central cached SFX service, route-aware music contexts, lifecycle handling and TV ducking. Music and reporter-blip provenance still require confirmation/replacement.
Supabase REST API	Implemented; live verification required	Anonymous ownership, profile setup, sanitized public/private snapshots, GET/POST/PATCH/DELETE, validation, safe errors, and separate caches.
Android build	Classroom APK ready; production work remains	Branded adaptive launcher icon, app label, immersive fullscreen, release APK artifact, and responsive phone tests. Production signing/device regression remain.

Area	Status	Current accomplishment
Automated testing	Passing	155 tests cover generation, integrity, persistence/migration, information consistency, API/SQL intent, settings, responsive UI, and the ballot-to-report loop.
Documentation	Completed by this deliverable	One consolidated midterm document with architecture, flowchart, ERD, component flow, scope, objectives, and evidence-based progress report.

8.3 Major completed features

- Responsive illustrated Main Menu, How to Play, five city slots, Continue/Start behavior, Settings, and local persistence.
- Procedural City Brief and Chronicle grounded in selected city issues and indicators.
- Five-candidate procedural election with truth-first generation, consistent cross-system references, and no exposed best-candidate score.
- Candidate Hub, Civic Archive biographies and citations, promises, Campaign Trail, Debate, records/evidence, CityBook, News, TV, and Receipts Board.
- Game-styled election, winner reveal, complete Living City term, interactive overhead map, project/event systems, and final Term Report.
- Versioned Sembast save/load and migration, SharedPreferences settings, English/Filipino localization, accessibility options, and centralized audio infrastructure.
- Optional Supabase gallery with privacy selection, Explore/My Cities, player search, owner-only privacy/delete actions, and sanitized JSON models.
- Android launcher branding, immersive system UI behavior, current release artifact, and automated responsive-screen checks.

8.4 Features currently being refined

- Physical-device visual regression, touch ergonomics, audio loudness, lifecycle behavior, and performance on lower-end Android phones.
- Live Supabase verification against the intended V2 tables, Anonymous Sign-Ins, RLS policies, ownership, and all four city-snapshot request methods.
- Additional procedural text and visual assets for candidate media, issue files, and term events while preserving stable saved-version behavior.
- Fine layout consistency across less frequently visited modals, media readers, and small/large screen sizes.
- Audio provenance and replacement of any classroom-only music or reporter sound before public distribution.

8.5 Remaining work after the midterm

- Connect completed terms into multiple election cycles with inherited city state, election history, and end-of-run reflection.
- Expand long-term candidate/administration history, laws, promise tracking, and content anti-repetition across elections.
- Design moderation and richer comparisons before expanding the public gallery's social surface.
- Add recoverable accounts or cloud saves only if the project later requires cross-device identity; the current local-first model should remain authoritative.
- Configure private production signing, finalize licensed audio, optimize release size, and complete store/public-release requirements.
- Runtime AI presentation remains deferred. If added later, it must paraphrase structured truth only, have a deterministic fallback, and never decide outcomes or fabricate evidence.

8.6 Testing and stability status

Verification area	Current evidence	Boundary
Static analysis	No issues found on the current v1.0 branch on September 2, 2026.	Does not replace runtime/device testing.
Automated tests	155 tests passed, including domain, data, persistence, API/SQL intent, settings, and widget flows.	Mocked/local tests cannot prove the deployed Supabase dashboard state.
Gameplay loop	Widget tests cover the ballot, winner reveal, term events, report, save restore, and targeted navigation flows.	A full physical-device demonstration path should still be repeated before submission.
Android artifact	Debug and release APK files are present; native fullscreen and launcher resources compile.	Release uses debug signing and is suitable only for classroom/demo distribution.
Performance design	One raster map, bounded daily logic, capped marker widgets, cached audio, and saved event records reduce runtime work.	Low-end phone behavior requires manual confirmation.

8.7 Repository documentation inconsistencies found

Actual source code was prioritized where older reports disagreed. The current LivingCityEngine generationVersion is 4 even though an older architecture paragraph names version 2. GameSaveData.currentSchemaVersion is 8 even though an earlier candidate-system paragraph still mentions schema 7. The August release-readiness report says no Android SDK/APK was available, but the current workspace contains successfully built debug and release APK artifacts and a branded launcher icon. These are documentation-age differences; the current implementation and this report use the source and verified build state.

9. Summary

Resibo, Please currently demonstrates its intended educational core: the player must understand a fictional city, investigate five imperfect candidates through several information formats, make an independent electoral decision, and review how that decision interacts with institutions, budget pressure, public trust, city problems, projects, and external conditions over one simulated year.

The strongest technical characteristic is consistency. Versioned seeded generation creates structured truth, the information ecosystem presents that truth from different perspectives, local persistence prevents rerolls, and the Living City reuses saved event IDs across the map, Chronicle, News, television, CityBook, and final report. The optional Supabase feature is deliberately smaller than the local save and shares only a sanitized outcome summary.

For the midterm, the system is a complete one-election MVP with passing automated checks and an Android classroom artifact. The next priorities are physical-device and live-backend verification, media/audio licensing cleanup, and controlled expansion toward multiple connected elections rather than broadening the scope before the current loop is stable.

Appendix A — Selected System Screenshots

The following tracked screenshots are actual project captures and remain representative of the current Main Menu and pre-election City Brief. Older repository captures of the authored three-candidate ballot, manual four-phase term, and V1 gallery were intentionally excluded because those interfaces have been superseded.



Figure A1. Main Menu of Resibo, Please.



Figure A2. Pre-election City Brief.

Screenshot source: docs/week1/actual_screens in the current repository. No mock or fabricated application screen is presented in this appendix.